

BigQuery Avançat & Analytics Engineering

Yonadxandi Manriquez

P2P Pau Serrano

Nivel 1: Entorno e Ingesta Híbrida (Code-First)

Escenario:

El cuadro de mando tarda demasiado en cargar y el coste es inaceptablemente alto. Se ha identificado que el problema crítico son los informes que creen ventas con regiones geográficas.

Ex. 1: Consulta sobre Tabla no Optimizada (Diagnóstico)

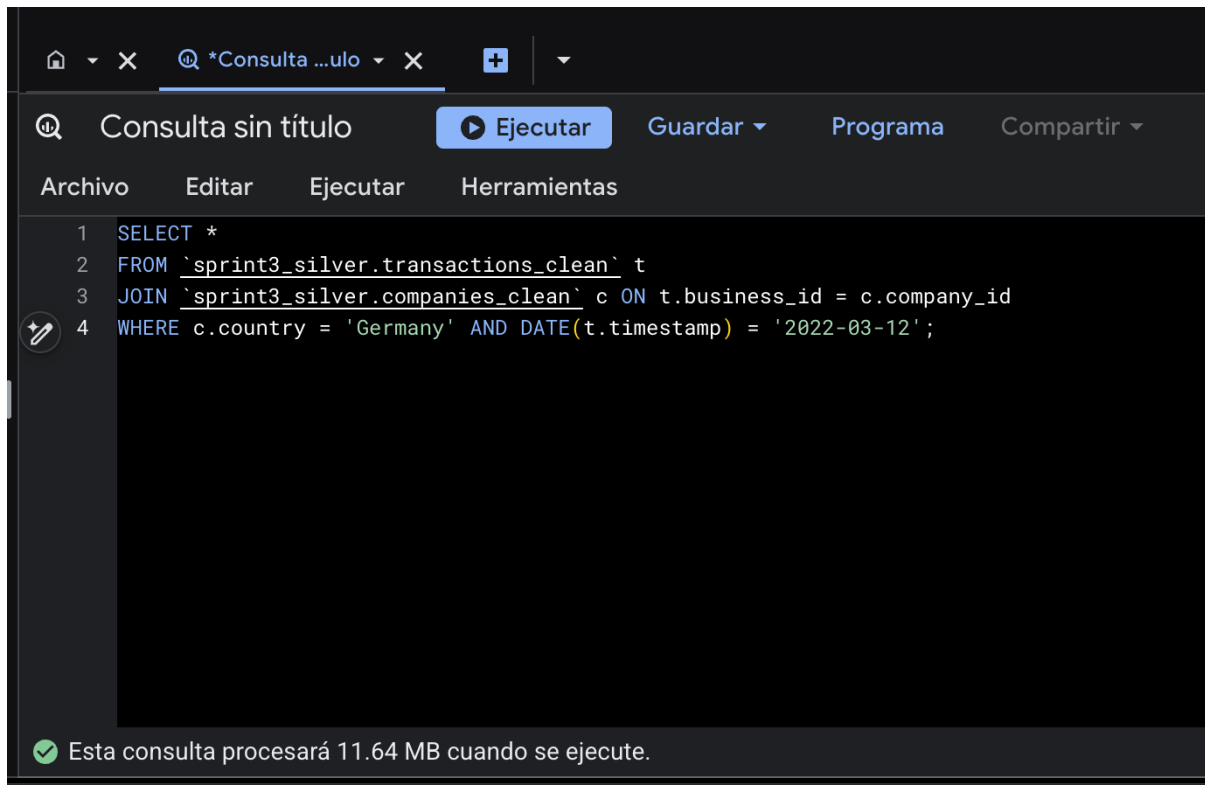
El Country Manager de Alemania necesita revisar urgentemente las transacciones del día 12 de marzo de 2022.

Tarea:

1. Escribe la consulta que une (JOIN) transacciones y compañías.
2. Filtra los resultados por la fecha indicada y el país "Germany".
3. Sin ejecutar la consulta, realiza un "Dry Run" (auditoria de costos).

Observación: Fíjate que BigQuery lea casi toda la tabla a pesar de pedir sólo un día (Full Table Scan).

=====



The screenshot shows the BigQuery console interface. At the top, there's a tab labeled '*Consulta ...ulo'. Below it, the title bar says 'Consulta sin título' with buttons for 'Ejecutar', 'Guardar', 'Programa', and 'Compartir'. A menu bar includes 'Archivo', 'Editar', 'Ejecutar', and 'Herramientas'. The main area contains a SQL query:

```
1 SELECT *
2 FROM `sprint3_silver.transactions_clean` t
3 JOIN `sprint3_silver.companies_clean` c ON t.business_id = c.company_id
4 WHERE c.country = 'Germany' AND DATE(t.timestamp) = '2022-03-12';
```

At the bottom, a status message with a green checkmark states: 'Esta consulta procesará 11.64 MB cuando se ejecute.'

Se escribe la consulta uniendo transacciones y companies. Antes que se ejecute, se observa que esta consulta leerá alrededor de 11.64 MB de datos, aunque se aplicó un filtro solicitando una fecha y un país, el BQ está leyendo todos los datos de la tabla.

La razón de ello es debido a que la tabla transaction clean no está organizada en alguna forma especial, o mejor dicho no tiene particiones, por lo que BigQuery no sabe en qué filas está ese día sin leerlas todas.

Ex. 2: Re-arquitectura y Optimización del Almacenaje (Partition & Cluster)

Escenario:

Después del diagnóstico del ejercicio anterior, hemos confirmado que filtrar la tabla de transacciones por fecha o por empresa provoca un Full Table Scan (lectura completa de la tabla), cosa que es ineficiente. Para solucionarlo en la capa de consumo (Gold), materializaremos una versión optimizada de la tabla de hechos.

Problema del sandbox: como utilizamos un entorno gratuito sin tarjeta de crédito, BigQuery borra automáticamente los datos de las particiones con más de 60 días de antigüedad. Dado que nuestros datos históricos son antiguos, si particionamos directamente la tabla, esta acabará quedando vacía. Para evitarlo, lo resolveremos en dos pasos: primero, actualizaremos automáticamente las fechas de los datos y, después, aplicaremos la optimización física.

Tarea:

Paso 1: Generación de Datos Recientes (Mocking Data)

Crea una tabla intermedia denominada `sprint3_silver.transactions_recent` a partir de la tabla

`sprint3_silver.transactions_clean`. Tu objetivo es mantener todas las columnas, pero sustituir el timestamp original por uno nuevo, generado aleatoriamente para que caiga dentro de los últimos 50 días.

Pista Técnica: Para hacerlo sin escribir todas las columnas a mano, puedes utilizar `SELECT`

`* EXCEPT(timestamp)`. Para calcular la fecha aleatoria, tendrás que restar una cantidad de días al instante actual.

Te serán muy útiles las funciones `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`, y la combinación `CAST(RAND() * 50 AS INT64)` para generar los días de desfase.

Paso 2: Creación de la Tabla Optimizada (Partitioning & Clustering)

Ahora, crea la tabla definitiva

`sprint3_gold.fact_transactions_optimized` a partir de los datos recientes que acabas de generar en `transactions_recent`. Has de

construir la sentencia DDL para configurar la tabla con las siguientes estrategias físicas de almacenaje:

- **Particionamiento (Partitioning):** Divide la tabla por la fecha del campo DATE(timestamp). Eso permitirá que las consultas que filtren por día (WHERE date = ...) sólo lean la partición necesaria.
- **Clustering:** Ordena los datos dentro de cada partición por business_id. Eso acelerará drásticamente los filtros para compañía y los cruces (JOINS) con la dimensión de empresas.

Entrega:

- El código SQL utilizado para el Paso 1.
- El código DDL (CREATE OR REPLACE TABLE...) utilizado en el Paso 2.
- Una captura de pantalla de los "Detalles" de la nueva tabla fact_transactions_optimized donde se pueda comprobar que está particionada y agrupada en clústers correctamente.

=====

Se realiza un PARTITION, que nos permitirá dividir por secciones, así el BQ, solo tendrá que acceder a esa sección.

Al aplicar un CLUSTER, se ordenan los datos por empresa (business_id), por lo tanto, al filtrar por empresa, se encuentran esos datos más rápido, sin revisar toda la sección.

Paso 1: Generación de Datos Recientes (Mocking Data)

Se copia la tabla transaction_clean, cambiando el timestamp por uno aleatorio de los últimos 50 días. Para excluir la columna que ya tenemos usamos EXCEPT para no duplicarla.

Se usa CURRENT_TIMESTAMP() para establecer el momento de ahora, la incluimos en TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL X DAY) para tomar el momento de ahora y restar X días. Esta consulta significa, que para cada fila de la tabla, BigQuery genera un número aleatorio distinto (entre 0 y 50) y resta esos días a la fecha de hoy.

Por eso cada transacción acaba con una fecha distinta, pero siempre dentro de los últimos 50 días (a fecha 16/06/2026)

```

1 CREATE OR REPLACE TABLE `sprint3-analytics-yonamanrique.sprint3_silver.transactions_recent`
2 AS
3 SELECT * EXCEPT(timestamp), TIMESTAMP_SUB(
4   CURRENT_TIMESTAMP(),
5   INTERVAL CAST(RAND() * 50 AS INT64) DAY
6 ) AS timestamp
7 FROM `sprint3-analytics-yonamanrique.sprint3_silver.transactions_clean`;

```

Se completó la consulta

Se comprueba en previsualización en la tabla `transaction_recent` dentro `sprint3_silver` que las fechas son de hace pocos días ((dentro de los últimos 50 días desde hoy)

Fila	transaction_id	card_id	busin...	amount	declined	product...	user_id	lat	longitude	timestamp
1	26904E68-8...	CcS-4857	b-2406	6.9	0	95	276	40.8074138...	-3.68159099...	2026-06-12 11:25:43.884717 UTC
2	2C8531E2-A...	CcS-4857	b-2466	482.91	0	61	276	40.8860417...	-3.67715503...	2026-06-09 11:25:43.884717 UTC
3	E69DE824-5...	CcS-4857	b-2346	264.24	0	47	276	40.1062409...	-3.34941710...	2026-04-27 11:25:43.884717 UTC

Resultados por página: 50 1 - 50 de 100000

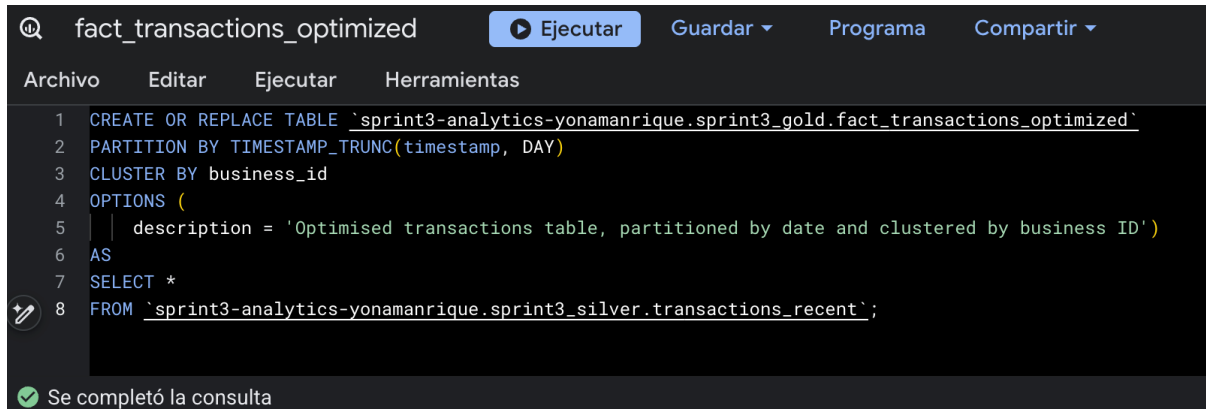
Paso 2: Creación de la Tabla Optimizada (Partitioning & Clustering)

Se crea `fact_transactions_optimized` en Gold, a partir de `transactions_recent`, pero esta vez con dos instrucciones especiales en el DDL: `PARTITION BY` y `CLUSTER BY`.

`PARTITION BY TIMESTAMP_TRUNC(timestamp, DAY)` indica que esta tabla tiene secciones separado uno por día.

Se usa `TIMESTAMP_TRUNC(timestamp, DAY)` y no `DATE(timestamp)` como en ejercicios anteriores para mantener el `TIMESTAMP` original sin cambiar los datos manteniendo la fecha sin la hora.

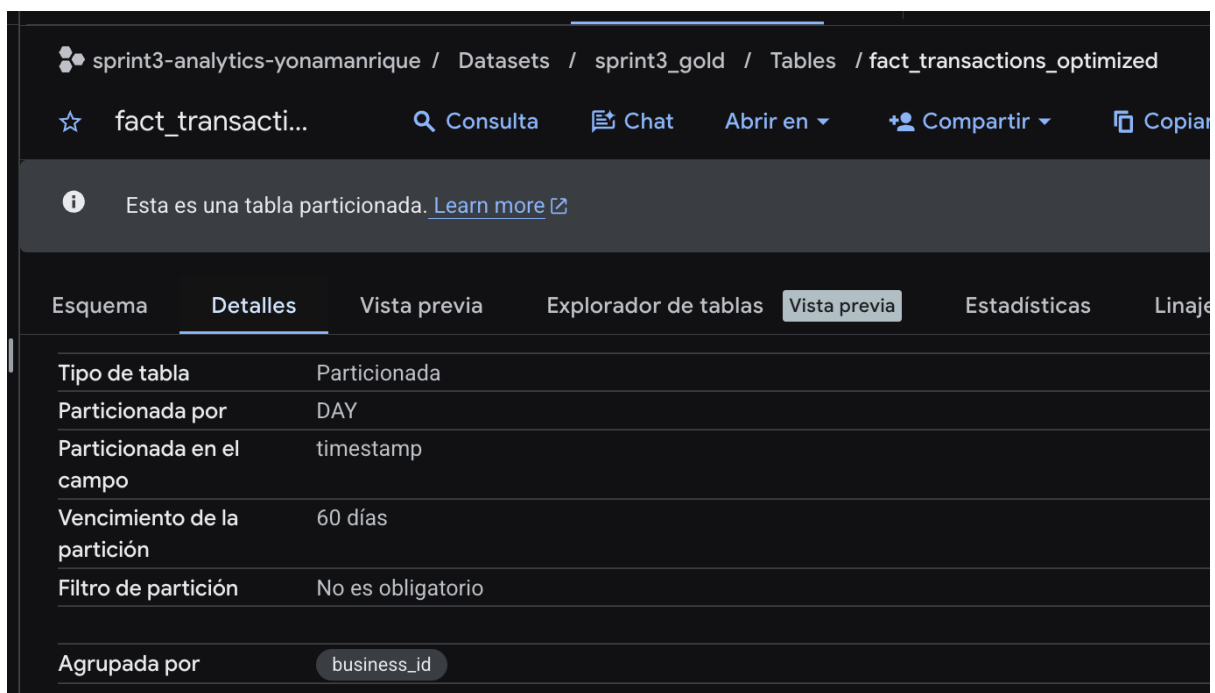
Esta consulta lo que hace es copiar todos los datos de `transaction_recent` guardándola de forma organizada (por día, y dentro de cada día por empresa), en lugar de guardarlos sin ningún orden como antes.



```
fact_transactions_optimized [Ejecutar] [Guardar] [Programa] [Compartir]
Archivo Editar Ejecutar Herramientas
1 CREATE OR REPLACE TABLE `sprint3-analytics-yonamanrique.sprint3_gold.fact_transactions_optimized`
2 PARTITION BY TIMESTAMP_TRUNC(timestamp, DAY)
3 CLUSTER BY business_id
4 OPTIONS (
5   | description = 'Optimised transactions table, partitioned by date and clustered by business ID')
6 AS
7 SELECT *
8 FROM `sprint3-analytics-yonamanrique.sprint3_silver.transactions_recent`;

Se completó la consulta
```

Comprobamos



sprint3-analytics-yonamanrique / Datasets / sprint3_gold / Tables / fact_transactions_optimized

fact_transacti... Consulta Chat Abrir en Compartir Copiar

Esta es una tabla particionada. [Learn more](#)

Esquema	Detalles	Vista previa	Explorador de tablas	Vista previa	Estadísticas	Linaje
Tipo de tabla	Particionada					
Particionada por	DAY					
Particionada en el campo	timestamp					
Vencimiento de la partición	60 días					
Filtro de partición	No es obligatorio					
Agrupada por	business_id					

Encontramos que se agrupo (CLUSTER) correctamente por `business_id` y está particionada por Day

Ex. 3: La Prueba del Algodón (Benchmark)

Escenario:

Ya has creado la tabla `sprint3_bold.fact_transactions_optimized` con las estrategias de Partición y Clustering. Ahora toca demostrar al equipo la mejora de rendimiento y el ahorro de costos que has logrado. Simularemos una petición típica de negocio: "Queremos ver todas las transacciones de los últimos 30 días".

Tarea:

El objetivo es claro y directo: aplicar exactamente la misma consulta a dos tablas diferentes y comparar el costo computacional.

Construye una consulta SQL que seleccione todas las columnas (`SELECT *`) y filtre los datos de los últimos 30 días.

NO ejecutes las consultas aún. En este ejercicio sólo utilizaremos el validador de BigQuery para visualizar los Bytes procesados de manera gratuita.

Haz la comparativa siguiendo estos dos pasos:

- Paso 1 (Tabla no optimizada): Apunta tu consulta en la tabla `sprint3_silver.transactions_recent`.

Observa el validador y haz una captura de pantalla de los bytes que procesaría.

- Paso 2 (Tabla optimizada): Sin cambiar nada de la lógica de tu consulta, modifica sólo el `FROM` para que ahora apunte a la tabla particionada `sprint3_gold.fact_transactions_optimized`. Observa como cambia el validador y hazla segunda captura.

Entrega:

- El código SQL base que has diseñado.
- Las dos capturas de pantalla donde se vea la diferencia de "Bytes procesados" en el validador.
- Un breve texto calculando cual es el % de ahorro que has logrado (compara los bytes del Paso 1 respecto a los del Paso 2).

=====

Código base:

```
SELECT *  
  
FROM `sprint3-analytics-  
yonamanrique.sprint3_silver.transactions_recent`  
  
WHERE DATE(timestamp) BETWEEN DATE_SUB(CURRENT_DATE(),  
INTERVAL 30 DAY) AND CURRENT_DATE();
```

Tabla sin particionar procesará **11.63 MB**

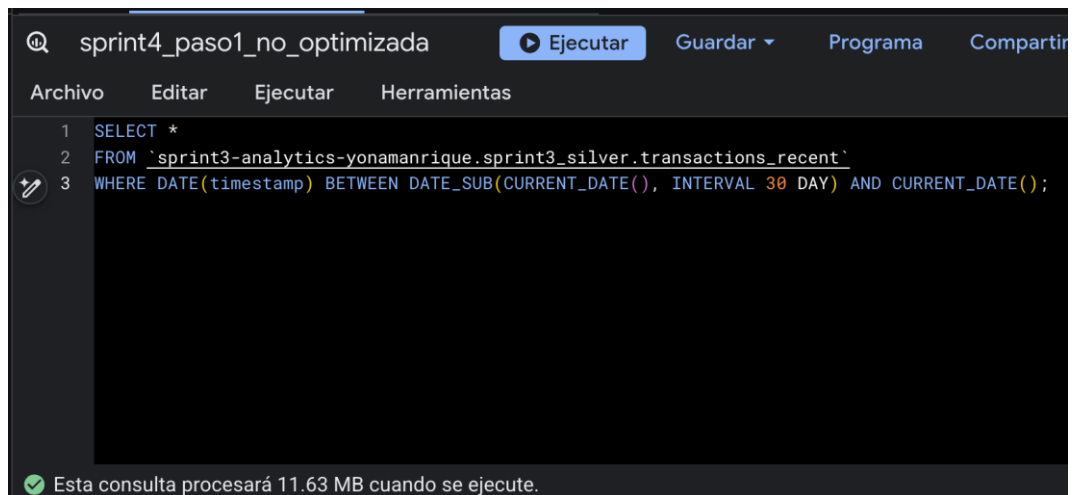
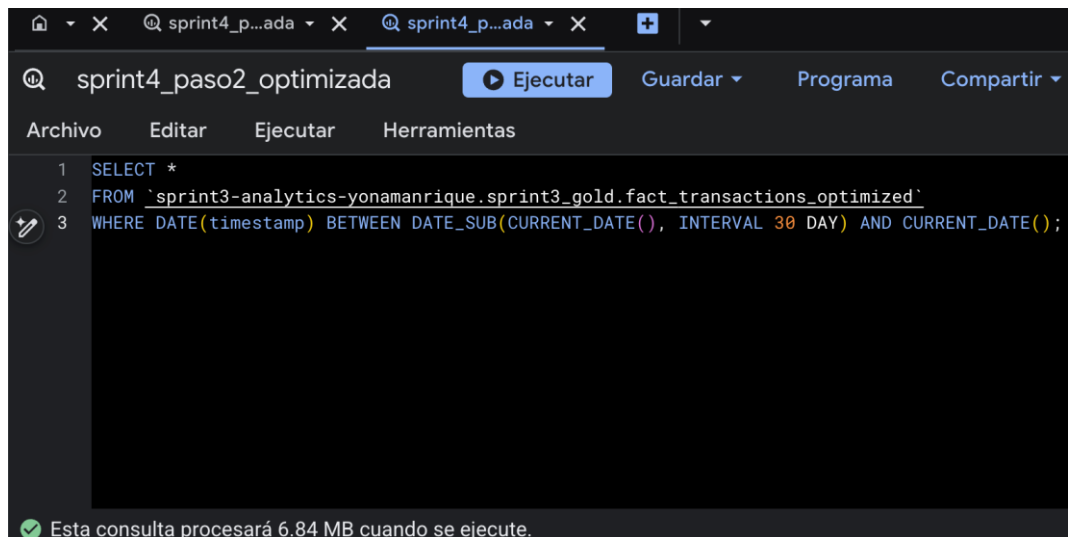


Tabla optimizada procesará **6.84 MB**



Fórmula = (Diferencia/valor original)/100

$(11.63 - 6.84) / 11.63 * 100$

$11.63 - 6.84 = 4.79$ MB menos que procesar

Ahorro = 41.2%

Al solicitar los últimos días a BQ de la tabla particionada, solo accedió a la información contenida en estas particiones de 30 días específicos, lo que se traduce en menos dinero en la factura de Google Cloud pagando solo que procesamos.

Ex. 4: Smart Caching (Vistas Materializadas)

Escenario:

El Director General tiene un cuadro de mando que muestra las "Ventas Totales por Día". Él (y 50 managers más) refrescan este gráfico constantemente.

Problema:

Si utilizas una consulta normal o una Vista standard, cada vez que alguien refresca la página, BigQuery recalcula la suma de millones de filas. Es lento y estás pagando por el mismo cálculo una y otra vez.

Solución:

Utilizar una Vista Materializada. Esta tecnología guarda el resultado del cálculo en el caché y sólo la actualiza si entran datos nuevos (incremental).

Tarea:

- Crea una Vista Materializada denominada `sprint3_gold.mv_daily_sales` que muestre las Ventas Totales por Día.

Entrega:

- Código para Crear la Vista Materializada.
- Haz una consulta que muestre la vista creada y haz una captura de pantalla que muestre en su conjunto la vista materializada creada, con la consulta ejecutada y los bytes procesados.

=====

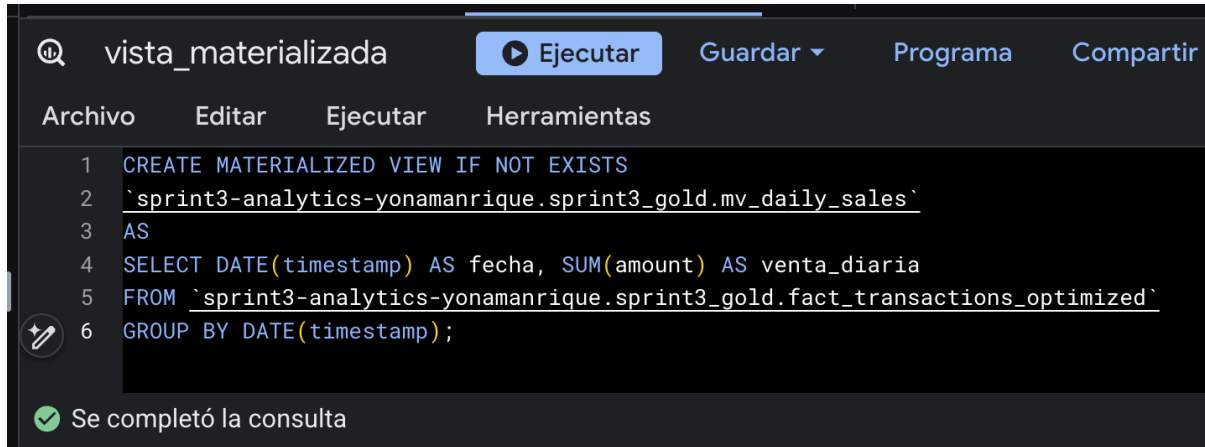
Para no ejecutar una consulta desde cero, aunque la tengamos guardada como vista, se usó la vista materializada la cual tiene la capacidad de guardar el resultado calculado, recalculado solo si hay datos nuevos ¹.

Por lo tanto, al consultar una vista de este tipo, el coste en bytes se reduce en comparación que si se realiza una consulta directamente en la tabla (fact_transactions_optimized con un GROUP BY).

Código:

```
CREATE MATERIALIZED VIEW IF NOT EXISTS
`sprint3-analytics-yonamanrique.sprint3_gold.mv_daily_sales`
AS
SELECT DATE(timestamp) AS fecha, SUM(amount) AS venta_diaria
FROM `sprint3-analytics-yonamanrique.sprint3_gold.fact_transactions_optimized`
GROUP BY DATE(timestamp);
```

Nombre de la vista materializada: mv_daily_sales



Consulta:

```
SELECT *
```

```
FROM `sprint3-analytics-  
yonamanrique.sprint3_gold.mv_daily_sales`;
```

Resultados de la consulta				Chat	View all jobs	Guardar los resultados ▼
Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución	
Fila	fecha	venta_diaria				
1	2026-04-27	263156.4299999...				
2	2026-04-30	529360.1500000...				
3	2026-05-01	544173.2200000...				
4	2026-06-12	521933.1500000...				
5	2026-05-20	495857.9400000...				
6	2026-06-03	524168.6500000...				
7	2026-06-01	501354.9400000...				
8	2026-06-13	499136.6100000...				
9	2026-05-27	505291.3000000...				
10	2026-06-11	507301.8500000...				
11	2026-05-17	534172.9800000...				

Resultados por página: 50 ▼ 1 - 50 de 51

Resultados de la consulta	
Información del trabajo	Resultados
ID de trabajo	sprint3-analytics-yonamanrique:EU.bquxjob_266
Usuario	yonamanriquez@gmail.com
Ubicación	EU
Hora de creación	17 jun 2026, 12:16:47 p.m. UTC+2
Hora de inicio	17 jun 2026, 12:16:47 p.m. UTC+2
Hora de finalización	17 jun 2026, 12:16:48 p.m. UTC+2
Duración	0 s
Bytes procesados	816 B
Bytes facturados	10 MB
Milisegundos de ranura	56
Prioridad del trabajo	INTERACTIVE
Usar SQL heredado	false
Tabla de destino	Tabla temporal

La consulta nos devuelve 51 filas y 816 B procesados, es menor a un 1 KB, lo que indica que no realizó el BQ un recalcule en la suma de todas las filas, solo devolvió el resultado que ya tenía guardado y calculado prácticamente "sin coste de procesamiento".

En cuanto a los bytes facturados que fueron 10MB. BQ cobra una cuota mínima de facturación, aunque se procese un mínimo por consulta como es este caso, aun así la cuota cobra un mínimo de 10 MB.

Nivel 2: SQL Analítico Avanzado

Escenario:

La dirección comienza a hacer preguntas que no se resuelven con un simple GROUP BY. Necesitamos análisis de tendencias y comportamiento de usuario.

Ex 1: Perfilado de Clientes VIP (Métricas Agregadas con CTEs)

Escenario:

El equipo de Marketing quiere analizar el comportamiento de nuestros mejores clientes para diseñar la estrategia del próximo año. Definimos "VIP" como aquellos con un gasto acumulado superior a 500 €. Necesitan un informe que, para cada VIP, muestre su nombre, contacto y su patrón de compra: cuántas veces ha comprado, cuánto gasta de media y cuál fue su compra récord.

Tarea:

1. Crea una CTE llamada VIP_Stats que agrupe por usuario y calcule:

El Gasto Total (SUM).

La Cantidad de Transacciones (COUNT).

El Ticket Medio (AVG), redondeado a 2 decimales.

La Compra Máxima (MAX).

Filtro: Mantiene sólo aquellos cuyo Gasto Total sea > 500.

2. Crea la CTE con users_combined para obtener los datos personales.

Requisitos de Salida:

Columnas: user_id, nombre_completo, email, num_compras, ticket_medio, max_compra, total_gastat.

Ordenado por total_gastado descendente.

Se crea un CTE para organizar por agrupaciones los cálculos que se solicitan usando `fact_transactions_optimized` que es nuestra tabla que anteriormente ya estaba particionada y optimizada.

Consulta del CTE:

```
WITH VIP_stats AS (  
  SELECT  
    user_id,  
    ROUND(SUM(amount), 2) AS total_gastado,  
    COUNT(*) AS num_compras,  
    ROUND(AVG(amount), 2) AS ticket_medio,  
    ROUND(MAX(amount), 2) AS max_compra  
  FROM `sprint3-analytics-  
yonamanrique.sprint3_gold.fact_transactions_optimized`  
  GROUP BY user_id  
  HAVING SUM(amount) > 500  
)
```

En la consulta se usa `HAVING SUM(amount) > 500` y no el alias `total_gastado`, porque BigQuery no permite referenciar alias en la cláusula `HAVING`.

A continuación, se cruzan los cálculos con los datos personales solicitados en el ejercicio.

```
SELECT  
  u.user_id,  
  CONCAT(u.name, ' ', u.surname) AS nombre_completo,  
  u.email,  
  v.num_compras,  
  v.ticket_medio,  
  v.max_compra,  
  v.total_gastado  
FROM  
  `sprint3-analytics-yonamanrique.sprint3_silver.users_combined` u  
JOIN VIP_stats v ON u.user_id = v.user_id  
ORDER BY v.total_gastado DESC;
```

Se usó `Concat` para juntar el nombre y el apellido en una sola columna. Como ya se ha filtrado aquellos Vip que han gastado más de 500 €, usamos un `JOIN` normal en vez de un `LEFT JOIN`, esto para evitar que aparezcan los NO-VIP con columnas vacías (Null).

```

clientes_vip  Ejecutar  Guardar  Programa  Compartir
Archivo  Editar  Ejecutar  Herramientas

1  WITH VIP_stats AS (
2    SELECT
3      user_id,
4      ROUND(SUM(amount), 2) AS total_gastado,
5      COUNT(*) AS num_compras,
6      ROUND(AVG(amount), 2) AS ticket_medio,
7      ROUND(MAX(amount), 2) AS max_compra
8    FROM `sprint3-analytics-yonamanrique.sprint3_gold.fact_transactions_optimized`
9    GROUP BY user_id
10   HAVING SUM(amount) > 500
11 )
12 SELECT
13   u.user_id,
14   CONCAT(u.name, ' ', u.surname) AS nombre_completo,
15   u.email,
16   v.num_compras,
17   v.ticket_medio,
18   v.max_compra,
19   v.total_gastado
20 FROM `sprint3-analytics-yonamanrique.sprint3_silver.users_combined` u

```

Se completó la consulta

Resultados de la consulta							
Información del trabajo							
Resultados							
user_id	nombre_completo	email	num_compras	ticket_medio	max_compra	total_gastado	
289	Dxwgi Hwcru	dxwgi.hwcru@example.com	94	441.35	554.71	41486.54	
318	Bnyr Astuw	bnyr.astuw@example.com	91	443.39	615.51	40348.5	
455	Fwfil Phylgmhnd	fwfil.phylgmhnd@example.com	66	522.59	724.44	34491.27	
417	Fqatil Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	78	410.46	675.78	32016.11	
185	Molly Gilliam	donec@outlook.couk	110	275.75	659.31	30333.03	
285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51	
193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05	
454	Sfzzoh Xgyfridxs	sfzzoh.xgyfridxs@example.com	81	300.67	604.9	24353.91	
349	Yxxcjk Fnceqbkxmt	yxxcjk.fnceqbkxmt@example.c...	53	443.48	675.05	23504.47	
317	Qlkcvc Nvficfqdh	qlkcvc.nvficfqdh@example.com	54	351.69	600.77	18991.45	

Resultados por página: 50 1 - 50 de 5000

Dato interesante para resaltar. La consulta devuelve 5000 registros, que es el total de usuarios, lo que significa que todos los usuarios han gastado más de 500€ en total, quedando todos dentro del filtro VIP. No quiere decir, que el filtro de la consulta no funcione, solo que en este dataset no hay "no-VIP" que mostrar como contraste.

Ex 2: Análisis de Tendencias (Window Functions sobre Vistas)

Escenario:

La Dirección Financiera quiere monitorear la "velocidad de ventas" diaria. Necesitan un informe que compare el rendimiento de cada día contra el día anterior para detectar caídas bruscas o picos de crecimiento (Day-over-Day Growth).

Estrategia de Optimización:

Para este cálculo NO debes ir a la tabla de hechos original ni recalcular sumas. Como buen Data Engineer, debes utilizar la Vista Materializada `sprint3_gold.mv_daily_sales` que creaste en el ejercicio anterior. Esto garantiza que el análisis de tendencias sea instantáneo y no consuma recursos de cómputo innecesarios.

Tarea:

Crea una consulta sobre la vista materializada que genere un informe con las siguientes 4 columnas:

- 1.Fecha: La fecha de venta.
- 2.Ventas_Hoy: El total vendido ese día.
- 3.Ventas_Ayer: El total vendido el día anterior (usando funciones de ventana).
- 4.Diff_Percentual: El porcentaje de crecimiento o decrecimiento respecto al día anterior, redondeado a 2 decimales.

Pista Técnica:

Utiliza la función `LAG(campo) OVER (ORDER BY data)` para leer el valor de la fila anterior sin necesidad de realizar JOINS complejos.

=====

El objetivo de usar una vista materializada en lugar de la original es en resumen una estrategia de optimización. Para los datos, se usa mv_daily_sales en lugar de fact_transactions_optimized para no recalcular la suma de millones de filas en cada consulta.

Creamos la CTE para traer las ventas_hoy, ventas _ayer. Usamos en la consulta LAG, para poder mirar justo una posición hacia atrás y ordenar todo por fecha y para cada fila traer el valor de venta diaria de 1 fila hacia atrás o del día anterior.

```
WITH daily_sales_temp AS (
  SELECT
    fecha,
    venta_diaria AS ventas_hoy,
    LAG(venta_diaria, 1) OVER (ORDER BY fecha) AS ventas_ayer
  FROM `sprint3-analytics-
yonamanrique.sprint3_gold.mv_daily_sales`
);
```

Se calcula el porcentaje de diferencia entre hoy y ayer, después esa diferencia se divide entre el valor de ayer con el objetivo de obtener la proporción de cambio.

Para esta división se usa SAFE_DIVIDE. Se utiliza esta estructura (SAFE_DIVIDE) porque si en algún momento las ventas de ayer fueran cero o null, una división normal rompería toda la consulta con un error, con esto se evita que, si los valores no se pueden dividir, devuelva null en lugar de error y la consulta siga funcionando para el resto de las filas.

Al final convertimos en porcentaje multiplicando por 100

```
WITH daily_sales_temp AS (
  SELECT
    fecha,
    venta_diaria AS ventas_hoy,
    LAG(venta_diaria, 1) OVER (ORDER BY fecha) AS ventas_ayer
  FROM `sprint3-analytics-
yonamanrique.sprint3_gold.mv_daily_sales`
)
SELECT
  fecha,
  ROUND(ventas_hoy, 2) AS ventas_hoy,
  ROUND(ventas_ayer, 2) AS ventas_ayer,
```

```

    ROUND(SAFE_DIVIDE(ventas_hoy - ventas_ayer, ventas_ayer) * 100,
2) AS diff_percentual
FROM daily_sales_temp
ORDER BY fecha DESC;

```

SAFE DIVIDE Ejecutar Guardar Programa Compartir

Archivo Editar Ejecutar Herramientas

```

1 WITH daily_sales_temp AS (
2   SELECT
3     fecha,
4     venta_diaria AS ventas_hoy,
5     LAG(venta_diaria, 1) OVER (ORDER BY fecha) AS ventas_ayer
6   FROM `sprint3-analytics-yonamanrique.sprint3_gold.mv_daily_sales`
7 )
8 SELECT
9   fecha,
10  ROUND(ventas_hoy, 2) AS ventas_hoy,
11  ROUND(ventas_ayer, 2) AS ventas_ayer,
12  ROUND(SAFE_DIVIDE(ventas_hoy - ventas_ayer, ventas_ayer) * 100, 2) AS diff_percentual
13 FROM daily_sales_temp
14 ORDER BY fecha DESC;

```

Se completó la consulta

SAFE DIVIDE Ejecutar Guardar Programa Compartir

Archivo Editar Ejecutar Herramientas

```

1 WITH daily_sales_temp AS (
2   SELECT
3     fecha,
4     venta_diaria AS ventas_hoy,
5     LAG(venta_diaria, 1) OVER (ORDER BY fecha) AS ventas_ayer

```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Reciente	Resultados de la consulta					Chat	V
	Información del trabajo	Resultados	Visualización	JSON	Detalle		
11:20 a.m.	Fila	fecha	ventas_hoy	ventas_ayer	diff_percentual		
WITH daily_sale...	1	2026-06-16	254406.94	505479.57	-49.67		
	2	2026-06-15	505479.57	519985.02	-2.79		
	3	2026-06-14	519985.02	499136.61	4.18		
	4	2026-06-13	499136.61	521933.15	-4.37		
	5	2026-06-12	521933.15	507301.85	2.88		
	6	2026-06-11	507301.85	505557.01	0.35		
	7	2026-06-10	505557.01	521222.64	2.02		

Los resultados muestran las fluctuaciones, por ejemplo, en 2026-06-16 (la fecha más reciente, hoy) tuvo ventas de 254,406.94€, comparado con 505,479.57€ el día anterior. Eso es una caída del -49.67%, casi la mitad.

Resultados de la consulta						Chat	View all jobs
Información del trabajo		Resultados	Visualización		JSON	Detalles de la ejecución	
Fila	fecha	ventas_hoy	ventas_ayer	diff_percentual			
51	2026-04-27	263156.43	null	null			

Para comprobar se muestra la última fila, al ser la fecha mas antigua, aparecen en ventas_ayer null, porque no existe un día anterior para comparar. Esto confirma que el LAG está funcionando correctamente dejando un espacio cuando no hay datos.

Ex 3: Totales Acumulados (Running Totales sobre Vistas)

Escenario:

El Director Financiero (CFO) necesita visualizar la curva de crecimiento anual. No le interesa tanto lo que se vendió hoy, sino cuánto llevamos facturado en total desde el inicio del año hasta la fecha (YTD - Year To Date).

Tarea:

Utilizando la vista materializada mv_daily_sales (para no recalcular agregaciones base), genera un informe con tres columnas:

- 1.Fecha.
- 2.Ventas del Día: Redondeado a 2 decimales.
- 3.Ventas Acumuladas YTD: Una suma progresiva de las ventas que debe reiniciarse cada 1 de Enero. El resultado final debe estar redondeado a 2 decimales.

Pista Técnica:

Para conseguir el reinicio anual, necesitas dividir la ventana usando PARTITION BY EXTRACT(YEAR FROM data). La estructura completa sería: SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

=====

Se estructura la consulta:

SELECT

```

    fecha,
    ROUND(venta_diaria, 2) AS ventas_del_dia,
    ROUND (
        SUM(venta_diaria) OVER (
            PARTITION BY EXTRACT(YEAR FROM fecha)
            ORDER BY fecha
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ), 2
    ) AS ventas_acumuladas_ytd

```

```
FROM `sprint3-analytics-yonamanrique.sprint3_gold.mv_daily_sales`  
ORDER BY fecha ASC;
```

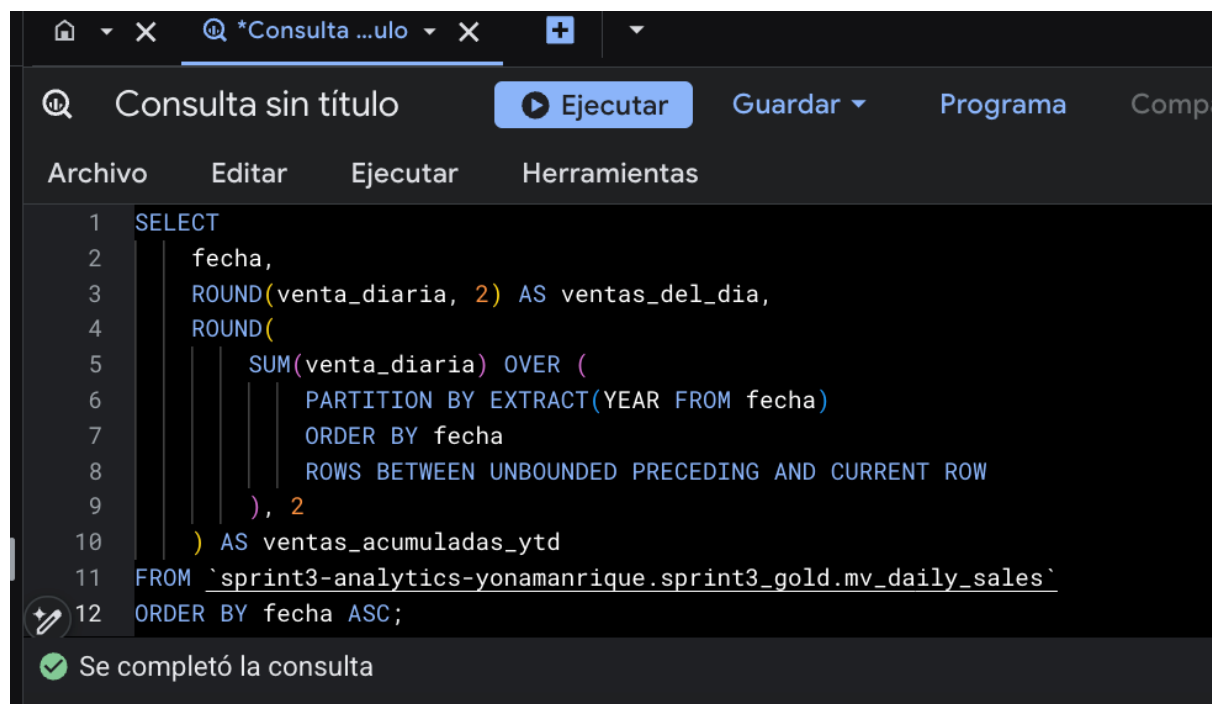
La consulta se construye con un PARTITION qué como se ha visto en ejercicios anteriores divide los datos en grupo y hace los cálculos separados por cada grupo.

EXTRACT(YEAR FROM fecha) saca solo el año de la fecha.

Entonces PARTITION BY EXTRACT(YEAR FROM fecha) se usa para agrupar por año y dentro de cada año, reinicie el conteo a cero cada 1 de enero.

Con ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW se utiliza para definir hasta donde mirar hacia atrás para hacer la suma acumulada. (Para mirar al principio del grupo UNBOUNDED PRECEDING, hasta el final CURRENT ROW)

Se añade al final ORDER BY fecha ASC, esto es para que la consulta devuelva las filas en un orden visual en que las fechas aparezcan en orden cronológico de arriba abajo, para una correcta interpretación de la curva de crecimiento que se solicita.



The screenshot shows a BigQuery console window with a dark theme. The title bar indicates the active tab is '*Consulta ...ulo'. The main toolbar contains buttons for 'Ejecutar' (Execute), 'Guardar' (Save), 'Programa', and 'Comp'. Below the toolbar is a menu bar with 'Archivo', 'Editar', 'Ejecutar', and 'Herramientas'. The SQL editor displays the following query:

```
1 SELECT  
2     fecha,  
3     ROUND(venta_diaria, 2) AS ventas_del_dia,  
4     ROUND(  
5         SUM(venta_diaria) OVER (  
6             PARTITION BY EXTRACT(YEAR FROM fecha)  
7             ORDER BY fecha  
8             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
9         ), 2  
10    ) AS ventas_acumuladas_ytd  
11 FROM `sprint3-analytics-yonamanrique.sprint3_gold.mv_daily_sales`  
12 ORDER BY fecha ASC;
```

At the bottom of the window, a status bar shows a green checkmark icon and the text 'Se completó la consulta' (The query was completed).

Resultados de la consulta					Chat	View all jobs	Guardar los resultados ▾
Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución		Gráfico de ej
Fila	fecha ▾	ventas_del_dia	ventas_acumulad...				
1	2026-04-27	263156.43	263156.43				
2	2026-04-28	531465.94	794622.37				
3	2026-04-29	523592.73	1318215.1				
4	2026-04-30	529360.15	1847575.25				
5	2026-05-01	544173.22	2391748.47				
6	2026-05-02	541456.63	2933205.1				
7	2026-05-03	509935.34	3443140.44				
8	2026-05-04	497437.69	3940578.13				
9	2026-05-05	507825.92	4448404.05				
10	2026-05-06	500100.50	4948504.55				
11	2026-05-07	500100.50	5448605.05				
12	2026-05-08	500100.50	5948705.55				
13	2026-05-09	500100.50	6448806.05				
14	2026-05-10	500100.50	6948906.55				
15	2026-05-11	500100.50	7449007.05				
16	2026-05-12	500100.50	7949107.55				
17	2026-05-13	500100.50	8449208.05				
18	2026-05-14	500100.50	8949308.55				
19	2026-05-15	500100.50	9449409.05				
20	2026-05-16	500100.50	9949509.55				
21	2026-05-17	500100.50	10449610.05				
22	2026-05-18	500100.50	10949710.55				
23	2026-05-19	500100.50	11449811.05				
24	2026-05-20	500100.50	11949911.55				
25	2026-05-21	500100.50	12450012.05				
26	2026-05-22	500100.50	12950112.55				
27	2026-05-23	500100.50	13450213.05				
28	2026-05-24	500100.50	13950313.55				
29	2026-05-25	500100.50	14450414.05				
30	2026-05-26	500100.50	14950514.55				
31	2026-05-27	500100.50	15450615.05				
32	2026-05-28	500100.50	15950715.55				
33	2026-05-29	500100.50	16450816.05				
34	2026-05-30	500100.50	16950916.55				
35	2026-05-31	500100.50	17451017.05				
36	2026-06-01	500100.50	17951117.55				
37	2026-06-02	500100.50	18451218.05				
38	2026-06-03	500100.50	18951318.55				
39	2026-06-04	500100.50	19451419.05				
40	2026-06-05	500100.50	19951519.55				
41	2026-06-06	500100.50	20451620.05				
42	2026-06-07	500100.50	20951720.55				
43	2026-06-08	500100.50	21451821.05				
44	2026-06-09	500100.50	21951921.55				
45	2026-06-10	500100.50	22452022.05				
46	2026-06-11	500100.50	22952122.55				
47	2026-06-12	500100.50	23452223.05				
48	2026-06-13	500100.50	23952323.55				
49	2026-06-14	500100.50	24452424.05				
50	2026-06-15	500100.50	24952524.55				
51	2026-06-16	254406.94	25901531.17				
					Resultados por página: 50 ▾ 1 – 50 de 51		

El resultado muestra que cada fila va sumando el valor del día anterior acumulado más las ventas de hoy.

Resultados de la consulta					Chat
Información del trabajo		Resultados	Visualización	JSON	
Fila	fecha ▾	ventas_del_dia	ventas_acumulad...		
51	2026-06-16	254406.94	25901531.17		

La fila final (2026-06-16, el día más reciente) muestra ventas del día de 254,406.94€, y el acumulado llega a 25,901,531.17€. Ese número es la suma de las 51 fechas completas, desde el primer día (2026-04-27) hasta hoy. Es el total mas grande de toda la tabla, tal y como se esperaba.

Ex 4: Fidelización y Valor del Cliente (Filtraje Avanzado)

Escenario:

El equipo de Marketing lanza la campaña "A la tercera va la vencida". Quieren enviar un regalo a los usuarios que hayan completado su tercera transacción. Sin embargo, el presupuesto del regalo no es fijo: depende de la "calidad" del cliente en sus inicios. Necesitan saber el Ticket Medio de estas 3 primeras compras para segmentar si envían un detalle básico o un regalo premium.

Tarea:

Genera un listado de los usuarios que han llegado (o superado) su tercera compra, mostrando:

1. Datos de usuario (user_id, nombre_completo, email).
2. La fecha y el importe exacto de la 3ª compra.
3. Media de las 3 primeras: La media de gasto de sus transacciones 1, 2 y 3.

Pista Técnica:

Tienes que utilizar la función ROW_NUMBER() para establecer el orden cronológico de las compras.

Tienes que utilizar la cláusula QUALIFY para filtrar.

Reto de Optimización: Busca la estrategia de menor coste computacional. Ten en cuenta que calcular medias sobre todo el historial histórico es costoso. Trate de filtrar las filas necesarias antes de realizar la agregación final.

=====

Para hacer más eficiente esta consulta se filtra las primeras 3 compras de cada usuario (filtro pronto), y después se calcula el promedio solo sobre esas pocas filas que quedan.

Se construye primero un CTE en donde para cada usuario, se numeran las compras en orden cronológico, y al final obtener solo con las 3 primeras antes de calcular el promedio.

```

WITH primeras_3_compras AS (
    SELECT
        user_id,
        timestamp,
        amount,
        ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp)
    AS num_compra
    FROM `sprint3-analytics-yonamanrique.sprint3_gold.fact_transactions_optimized`
    QUALIFY num_compra <= 3
)

```

Se calcula el promedio sobre las tres primeras compras de cada usuario y se obtiene solo la fila que sea exactamente la "3ra compra" (para mostrar su fecha e importe. Se trae también en nombre y email de users_combined.

```

SELECT
    u.user_id,
    CONCAT(u.name, ' ', u.surname) AS nombre_completo,
    u.email,
    DATE(p.timestamp) AS fecha_3a_compra,
    p.amount AS monto_3a_compra,
    ROUND(AVG(p.amount) OVER (PARTITION BY p.user_id), 2) AS
    prom_3_compras
FROM primeras_3_compras p
JOIN `sprint3-analytics-yonamanrique.sprint3_silver.users_combined`
u ON p.user_id = u.user_id
QUALIFY p.num_compra = 3;

```

Con `AVG(p.amount) OVER (PARTITION BY p.user_id)` se calcula el promedio, pero solo dentro del grupo de cada usuario (`PARTITION BY p.user_id`). Como ya se ha filtrado antes para obtener las 3 primeras compras, este promedio se calcula automáticamente sobre esas 3 filas, no sobre todas las filas registradas.

La alternativa que se piensa en primera instancia es calcular `AVG` sobre toda la tabla de transacciones y después filtrar.

Sin embargo, filtrando primero con `QUALIFY <= 3`, BigQuery solo procesa 3 filas por usuario en lugar de todas, optimizado y ahorrando en este proceso.

```

1 WITH primeras_3_compras AS (
2   SELECT
3     user_id,
4     timestamp,
5     amount,
6     ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp) AS num_compra
7   FROM `sprint3-analytics-yonamanrique.sprint3_gold.fact_transactions_optimized`
8   QUALIFY num_compra <= 3
9 )
10 SELECT
11   u.user_id,
12   CONCAT(u.name, ' ', u.surname) AS nombre_completo,
13   u.email,
14   DATE(p.timestamp) AS fecha_3a_compra,
15   p.amount AS monto_3a_compra,
16   ROUND(AVG(p.amount) OVER (PARTITION BY p.user_id), 2) AS prom_3_compras
17 FROM primeras_3_compras p
18 JOIN `sprint3-analytics-yonamanrique.sprint3_silver.users_combined` u ON p.user_id = u.user_id
19 QUALIFY p.num_compra = 3;

```

Se completó la consulta

`QUALIFY p.num_compra = 3` De las 3 filas que sobrevivieron por usuario (compra 1, 2 y 3), aquí se obtiene solo con la que es exactamente la compra número 3, para mostrar su fecha e importe exacto.

Resultados de la consulta							Chat	View all jobs	Guardar los resultados	Abrir en
Información del trabajo	Resultados			Visualización	JSON	Detalles de la ejecución		Gráfico de ejecución		
Id	user_id	nombre_completo	email	fecha_3a_compra	monto_3a_compra	prom_3_compras				
1	1006	Rbkocq Swkmynuj	rbkocq.swkmynuj@example.com	2026-05-01	303.75	122.28				
2	1035	Fqkihx Dmysrzuu	fqkihx.dmysrzuu@example.com	2026-05-15	112.09	211.97				
3	1042	Gfmdyb Kjyosqmc	gfmdyb.kjyosqmc@example.com	2026-05-02	377.79	273.82				
4	1112	Ljsbzb Cdtfpbib	ljsbzb.cdtfpbib@example.com	2026-04-28	213.56	158.63				
5	1208	Smufwc Rywckazm	smufwc.rywckazm@example.c...	2026-04-29	322.6	337.94				
6	1230	Leginv Pigidlq	leginv.pigidlq@example.com	2026-05-06	559.28	337.98				
7	1569	Hzfzsg Scismyhr	hzfzsg.scismyhr@example.com	2026-04-29	199.29	263.14				
8	16	Eric Roth	lorem.sit@yahoo.net	2026-05-02	263.78	217.39				
9	1610	Iqxlyz Xbgeozan	iqxlyz.xbgeozan@example.com	2026-05-05	167.66	157.99				
10	167	Sheila Dickerson	et.lacinia@hotmail.ca	2026-05-01	230.18	268.17				

Resultados por página: 50 1 – 50 de 5000

La consulta devuelve 5000 registros, igual que en el Ex.1. Eso confirma que todos los usuarios han llegado al menos a su 3ra compra, así que nadie quedó excluido del listado.

Nivel 3: Analytics Engineering (Arrays & Automatización)

Escenario:

Marketing necesita urgentemente un Ranking de Productos más Vendidos.

EX1: Desanidado y Aplanado de Datos (Unnesting)

Escenario:

Actualmente, en la tabla de transacciones, los productos vendidos están "escondidos" dentro de un Array (una lista de IDs). El equipo de ventas quiere analizar qué productos específicos se venden más, pero no pueden hacerlo si los datos están comprimidos en una lista. Necesitan una **Tabla Detallada (Flat Table)** donde cada fila represente un producto vendido, aunque esto signifique que el ID de la transacción se repita.

Tarea:

Crea la tabla **dim_transactions_flat** desnormalizando la información. Tienes que "explotar" el array de productos y cruzarlo con el catálogo maestro para obtener los nombres y precios individuales.

Pasos Técnicos:

1. Use **CROSS JOIN UNNEST(product_ids)** para transformar el Array en filas individuales.
2. Realiza un JOIN con la tabla products para obtener el nombre (name) y precio (price) de cada ítem.

Nota: Ten cuidado con los tipos de datos al realizar el JOIN (Array de INT64 vs Product_ID STRING).

3. No agrupe el resultado. Queremos ver el desglose línea por línea.

Resultado Esperado (Visualización):

Tu mesa final debe verse "plana", donde una transacción con 3 productos ocupa 3 filas:

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_A	Portàtil Bàsic	99.99
TXN1001	2025-12-08	155.97	SKU_B	Ratolí	12.99
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclat	45.50

Entrega:

Una captura de la Vista Previa de tu tabla en la que se vea el DDL ejecutada correctamente.

Una captura de pantalla en la que se vea el esquema de la nueva tabla.

Debido a que en el sprint 3 p.product_id se convirtió en STRING al limpiar products_clean y flat_product_id también en el mismo sprint se convirtió a INT64, se debe convertir este último a STRING al momento de construir el JOIN, de lo contrario BQ no los podrá comparar.

The screenshot shows the BigQuery console interface. At the top, there are tabs for 'Consulta sin título', 'Ejecutar', 'Guardar', 'Programa', and 'Compartir'. Below the tabs, there are menu items: 'Archivo', 'Editar', 'Ejecutar', and 'Herramientas'. The main area displays a SQL query with line numbers 1 through 12. The query creates or replaces a table named 'dim_transactions_flat' in the 'sprint3-gold' dataset. It selects columns from 'fact_transactions_optimized' and 'products_clean' tables, including transaction_id, fecha (timestamp), product_id, name, and price. The query uses a CROSS JOIN UNNEST and a JOIN with a CAST to STRING for product_id. At the bottom, a green checkmark and the text 'Se completó la consulta' indicate successful execution.

```

1 CREATE OR REPLACE TABLE `sprint3-analytics-yonamanrique.sprint3_gold.dim_transactions_flat`
2 AS
3 SELECT
4   t.transaction_id,
5   DATE(t.timestamp) AS fecha,
6   p.product_id,
7   p.name,
8   p.price
9 FROM `sprint3-analytics-yonamanrique.sprint3_gold.fact_transactions_optimized` t
10 CROSS JOIN UNNEST(t.product_ids) AS flat_product_id
11 JOIN `sprint3-analytics-yonamanrique.sprint3_silver.products_clean` p
12   ON p.product_id = CAST(flat_product_id AS STRING);

```

Se completó la consulta

The screenshot shows the BigQuery table schema view for the 'dim_transactions_flat' table. The breadcrumb navigation shows the path: 'sprint3-analytics-yonamanrique / Datasets / sprint3_gold / Tables / dim_transactions_flat'. The table name is 'dim_transactions_flat'. The schema is displayed in a table with columns: 'Nombre del campo', 'Tipo', 'Modo', 'Descripción', and 'Clave'. The columns listed are 'transaction_id' (STRING, NULLABLE), 'fecha' (DATE, NULLABLE), 'product_id' (STRING, NULLABLE), 'name' (STRING, NULLABLE), and 'price' (FLOAT, NULLABLE). At the bottom, there are buttons for 'Editar esquema', 'Ver políticas de acceso de fila', and 'Describir datos'.

Nombre del campo	Tipo	Modo	Descripción	Clave
transaction_id	STRING	NULLABLE	-	-
fecha	DATE	NULLABLE	-	-
product_id	STRING	NULLABLE	-	-
name	STRING	NULLABLE	-	-
price	FLOAT	NULLABLE	-	-

Editar esquema Ver políticas de acceso de fila Describir datos

La tabla contiene tres elementos en STRING y un FLOAT en price.

Aparece también la columna de fecha, la cual se puede añadir más tarde, pero conociendo el entorno de Looker Studio que suele ser sensible a las fechas, se decide integrarla en este apartado.

En esta tabla cada fila es un producto individual vendido, con su transacción, fecha, nombre y precio, indicando que el UNNEST funcionó correctamente desanidando cada elemento dentro del array.

Ex2: El Ranking de Ventas (Agregación Simple)

Escenario:

Ahora que has preparado la infraestructura y dispones de la mesa `dim_transactions_flat` (donde cada fila equivale a un producto vendido), generar informes analíticos es trivial. Ya no necesitas funciones complejas de desapego para responder a preguntas básicas de negocio.

Tarea:

Genera el Top 5 de productos más vendidos en la historia de la compañía.

Método:

Consulta directamente tu nueva tabla desnormalizada (`dim_transactions_flat`).

Al ser la tabla plana, simplemente necesitas una agrupación estándar (`GROUP BY`) por el nombre del producto y una cuenta (`COUNT`) de las filas.

Entrega:

El código SQL que genera el ranking.

Una captura del resultado con los 5 productos ganadores y cuántas unidades se han vendido de cada uno.

=====

```
SELECT
    product_id,
    name,
    COUNT(*) AS sold_units
FROM `sprint3-analytics-
yonamanrique.sprint3_gold.dim_transactions_flat`
GROUP BY product_id, name
ORDER BY sold_units DESC
LIMIT 5;
```

Para mejorar la consulta se realizó el GROUP BY por name y por product_id. La razón, si hubiese dos productos distintos con el mismo nombre, al agrupar solo por nombre, la consulta los fusionaría incorrectamente. Al incluir product__id se garantiza que cada producto sea único en el resultado.

Listado de productos

Resultados de la consulta

Chat

View all jobs

Guardar los resultados

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	product_id	name	sold_units		
1	52	riverlands the duel	2654		
2	29	Tully maester Taryl	2635		
3	21	duel Direwolf	2609		
4	16	the duel warden	2608		
5	66	mustafar jinn	2601		

Resultados por página: 501 – 5 de 5

Ex3: Automatización del Pipeline y Visualización

Escenario:

El informe estático del ejercicio anterior ha gustado mucho, pero el Director de Marketing necesita profesionalizarlo para la toma de decisiones diaria. Tiene dos requerimientos urgentes:

1. **Lógica Centralizada:** Quieren ver el precio con IVA (21%) desglosado. Sin embargo, dado que el impuesto puede cambiar por país o año, te prohíben poner el cálculo `precio * 1,21` directamente en el código ("Hardcoded"). Tienes que utilizar una función reutilizable.
2. **Frescor de Datos:** El tablero debe estar actualizado automáticamente cada mañana a las 07:00 AM, antes de la reunión diaria de ventas.

Tarea:

Tienes que evolucionar tu tabla `dim_transactions_flat` para incluir el cálculo de impuestos y automatizar su regeneración.

Pasos Técnicos:

1. **User Defined Functions (UDF):** Crea una función SQL persistente llamada `calculate_tax(amount)` que reciba un valor numérico y devuelva el resultado aplicando el 21% de impuesto.
2. Integración y Orquestación:

Modifica el código de creación de la tabla (del Ejercicio 1) para que utilice tu nueva función **`calculate_tax`** y genere una nueva columna: **`product_price_tax_inc`**.

3. Configura una Scheduled Query en BigQuery para que esta sentencia (CTAS) se ejecute automáticamente todos los días a las 07:00 AM.

Visualización (BI): Conecta Looker Studio a tu mesa `dim_transactions_flat` y crea un Dashboard: "Monitor de Rendimiento de Ventas" Entrega: El código SQL de la UDF. El código SQL actualizado de la creación de la mesa. Una captura de pantalla demostrando que la Scheduled Query está programada. Un vínculo del gráfico a Looker Studio.

Propuesta de Dashboard:

Titulo: Monitor de Rendimiento de Ventas

=====

Configura una página con 3 secciones clave:

Los "Big Numbers" (KPIs Generales):

En la parte superior, coloca tres tarjetas de resultados (Scorecards) para dar contexto inmediato.

1. KPI 1: Ingresos Totales (Con IVA)

- Métrica: `SUM(product_price_tax_inc)`
- Por qué: Muestra el volumen real de dinero que entra.

2. Transacciones Únicas (Pedidos)

- Métrica: `COUNT_DISTINCT(transaction_id)`

3. Transacciones Únicas (Pedidos)

- Campo Calculado: Crea un campo en Looker:

$$\frac{\text{SUM}(\text{product_price_tax_inc})}{\text{COUNT_DISTINCT}(\text{transaction_id})}$$

=====

Gráfica:

Evolución Temporal (Gráfico de Series Temporales) Bajo los KPIs, un gráfico de línea que ocupe todo el ancho.

Dimensión: `timestamp` (Desglosado por Fecha).

Métrica: `SUM(product_price_tax_inc)`.

Línea de Referencia (Opcional): Añade una media constante o una línea de tendencia.

Valor: Permite a Marketing ver si las ventas suben o bajan día a día, o si hubo un pico por alguna campaña específica.

=====

El "Top Productes" Enriquecido (Tabla con Mapa de Calor)

Olvidate del gráfico de barras simple. Utiliza una Tabla con Barras incrustadas o Mapa de Calor.

Dimensión: product_name.

Métrica 1: COUNT(*) → Renúmbalo en "Unidades Vendidas".

Métrica 2: SUM(product_price_tax_inc) → Renúmbalo en "Ingresos Totales".

Métrica 3: SUM(product_price_tax_inc) - SUM(product_unit_price) → Renúmbalo en "IVA Recaudado" (Esto demuestra el valor de tu UDF).

Estilo: En la configuración de estilo de la tabla, activa la opción "Muestra número como: Barra" o "Mapa de calor" para la columna de Ingresos.

=====

Paso1:

Se crea calculate__tax la cual recibe un precio y devuelve ese precio con el 21% de IVA añadido.

Código de la UDF:

```
CREATE OR REPLACE FUNCTION
`sprint3-analytics-
yonamanrique.sprint3_gold.calculate_tax`(price FLOAT64)
RETURNS FLOAT64
AS (ROUND(price * 1.21, 2));
```

Paso 2:

Una vez creada la UDF se actualiza la tabla dim_transactions_flat para que use la función calculate_tax y añada la columna product_price_tax_inc.

```
CREATE OR REPLACE TABLE `sprint3-analytics-
yonamanrique.sprint3_gold.dim_transactions_flat`
AS
SELECT
    t.transaction_id,
    DATE(t.timestamp) AS fecha,
    p.product_id,
    p.name,
    p.price,
    `sprint3-analytics-
yonamanrique.sprint3_gold.calculate_tax`(p.price) AS
product_price_tax_inc
FROM `sprint3-analytics-
yonamanrique.sprint3_gold.fact_transactions_optimized` t
CROSS JOIN UNNEST(t.product_ids) AS flat_product_id
JOIN `sprint3-analytics-yonamanrique.sprint3_silver.products_clean`
p
    ON p.product_id = CAST(flat_product_id AS STRING);
```

The screenshot shows the Google BigQuery interface. The breadcrumb navigation indicates the path: `sprint3-analytics-yonamanrique / Datasets / sprint3_gold / Tables / dim_transactions_flat`. The table name `dim_transactions_flat` is highlighted. The interface includes tabs for 'Esquema', 'Detalles', 'Vista previa' (selected), 'Explorador de tablas', 'Vista previa', 'Estadísticas', 'Linaje', 'Perfil de datos', and 'Calidad de los datos'. The 'Vista previa' tab displays a table with 13 rows and 8 columns. The columns are: `Fila`, `transaction_id`, `fecha`, `product_id`, `name`, `price`, `product_pric...`, and an unlabeled column. The data shows transactions for 'mustafar jinn' with a price of 2.12 and a product price of 2.57. The bottom of the interface shows 'Resultados por página: 50' and '1 - 50 de 253391'.

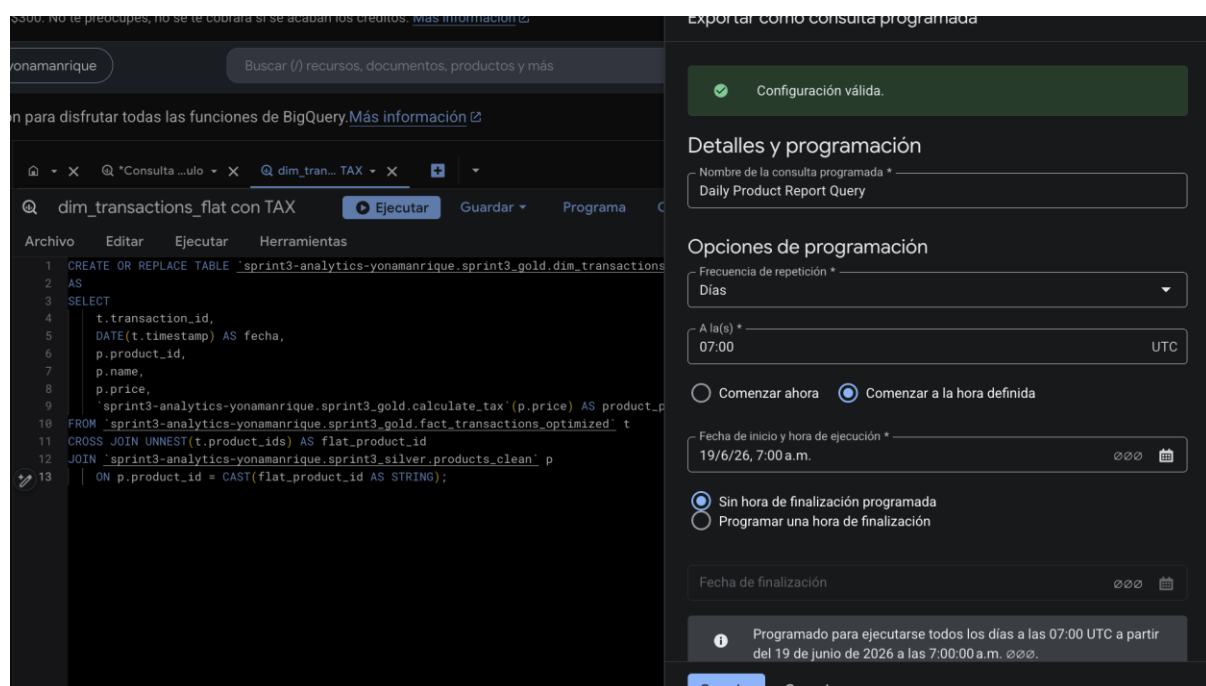
Fila	transaction_id	fecha	product_id	name	price	product_pric...	
1	CAF35BA5-E893-4761-95C6-77...	2026-04-27	66	mustafar jinn	2.12	2.57	
2	1E8C4656-68DD-4373-BA1F-48...	2026-04-27	66	mustafar jinn	2.12	2.57	
3	679E23E3-5D8D-4ACB-AD20-8D...	2026-04-27	66	mustafar jinn	2.12	2.57	
4	A3AC7630-4E18-48E9-B7CF-01...	2026-04-27	66	mustafar jinn	2.12	2.57	
5	EE606D01-5D7C-4C31-BBF4-A1...	2026-04-27	66	mustafar jinn	2.12	2.57	
6	9ECF2030-32EC-4E78-A7D7-10...	2026-04-27	66	mustafar jinn	2.12	2.57	
7	2DB0E2F7-D2D5-4B00-9307-57...	2026-04-27	66	mustafar jinn	2.12	2.57	
8	51176844-A68A-4B41-87CB-B6...	2026-04-27	66	mustafar jinn	2.12	2.57	
9	E11EA3E9-4D38-4599-9904-0E...	2026-04-27	66	mustafar jinn	2.12	2.57	
10	1C09E943-12CF-46C7-B681-6F...	2026-04-27	66	mustafar jinn	2.12	2.57	
11	AD3F3A46-9C11-49D8-8637-A9...	2026-04-27	66	mustafar jinn	2.12	2.57	
12	3E7261D1-353B-4FF6-92E0-63...	2026-04-27	66	mustafar jinn	2.12	2.57	
13	40342FE1-AAA6-4941-B8C9-F1...	2026-04-27	66	mustafar jinn	2.12	2.57	

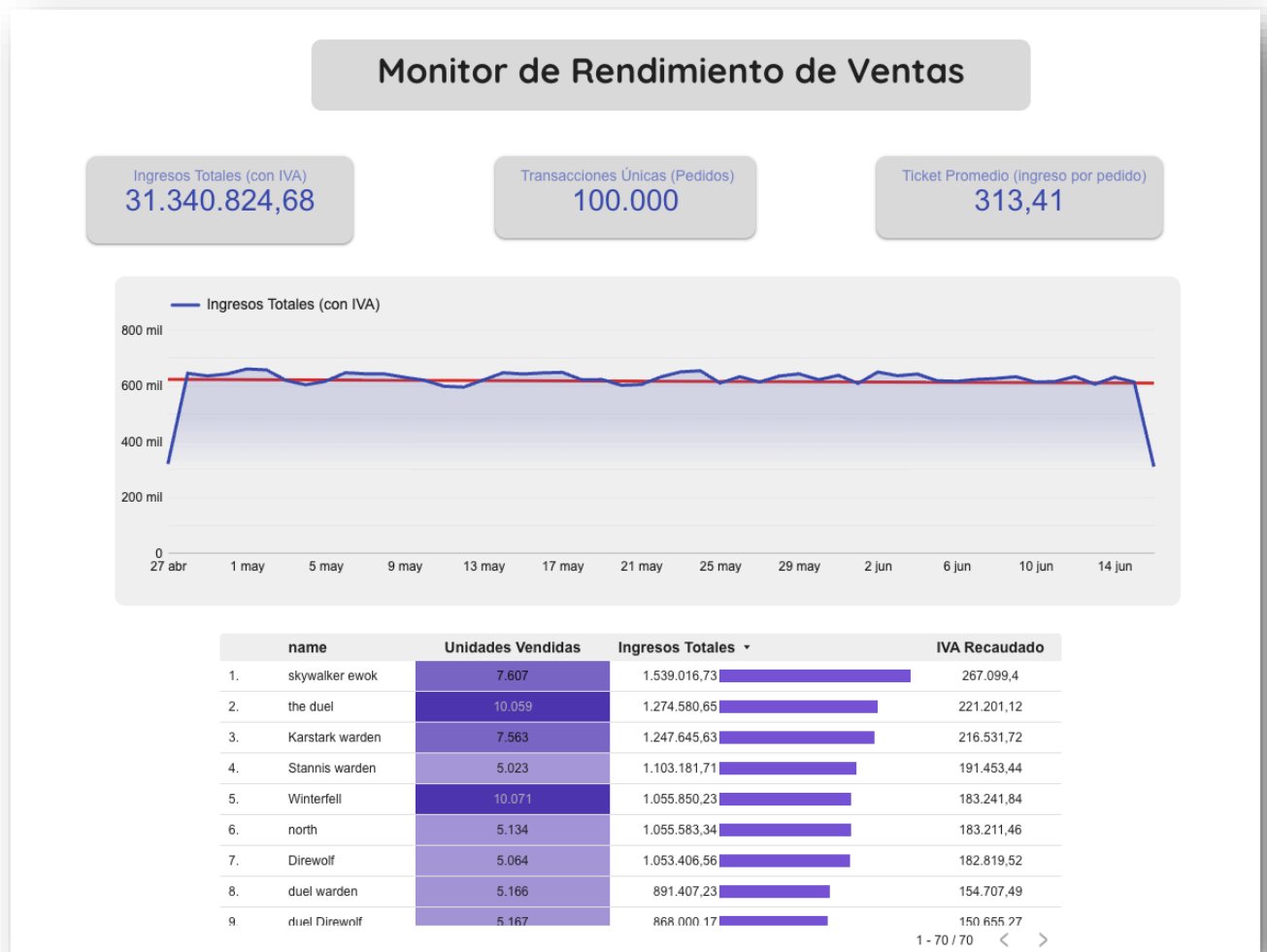
Para comprobar que la tabla se creó correctamente, se realizó una captura de pantalla de `dim_transaction_flat`. Se observa por ejemplo, en la primera fila `price: 2,12` al ver `product_price__tax__inc` su resultado de `2,57` se observa que se aplicó correctamente `1.21`, redondeando a dos decimales.

Parte 3: la Scheduled Query

Una aclaración. La cuenta gratuita (sandbox sin tarjeta de crédito) **no permite guardar Scheduled Queries**. BigQuery deja configurarla, pero al intentar guardarla, dará un error de facturación.

Se utilizó la UI para realizar la programación para que esta sentencia (CTAS) se ejecute automáticamente todos los días a las 07:00 AM.





El campo calculado de IVA recaudado que se observa en la tabla del dashboard, se calculó mediante la siguiente fórmula:
 $\text{SUM}(\text{product_price_tax_inc}) - \text{SUM}(\text{price})$, demostrando el valor de la UDF `calculate_tax`.

Link para acceder al dashboard

[Dashboard Motor de Rendimiento](#)

Link extenso:

<https://datastudio.google.com/reporting/7c496a69-7186-4c3f-b129-6deca1a007f5>

Este dashboard nos da una visión completa del rendimiento comercial de la empresa en tiempo real.

Los 3 grandes números nos dicen que la empresa ha generado 31,3 millones de euros en ingresos totales (IVA incluido), repartidos en 100.000 transacciones únicas. Esto nos da un ticket promedio de 313,41€ por pedido, lo que indica que los clientes gastan de media una cantidad considerable por compra.

Las ventas diarias se mantienen muy estables, alrededor de los 600.000€ diarios. La línea de tendencia roja confirma que no hay ni crecimiento ni caída significativa.

Se puede decir que el negocio es sólido y predecible. La bajada al final del gráfico corresponde al último día, que aún no tiene datos completos.

Los productos más vendidos en unidades son Winterfell y The Duel (más de 10.000 unidades cada uno). Sin embargo, en ingresos totales lidera Skywalker Ewok con 1,5 millones de euros, lo que sugiere que tiene un precio más alto. El IVA recaudado por producto nos permite ver exactamente cuánto tributa cada línea de producto.

Referencias

1. <https://aws.amazon.com/es/what-is/materialized-view/>